

# Delivery Tech API

## Guia Completo de Engenharia de Software, Arquitetura & Observabilidade

Do Código Limpo (Clean Code) à Produção: Spring Boot 3, Java 21 LTS, Docker, Prometheus, Grafana, Zipkin e Microsserviços

### [INSIGHT DE ENGENHARIA] A Virada de Chave: Coder vs. Engenheiro de Software

O **Coder** pensa apenas: *'a rota retornou 200 OK no meu localhost, meu trabalho acabou'*.

O **Engenheiro de Software** enxerga o sistema vivo em produção: *como auditar erros no meio de milhares de acessos simultâneos? O que fazer se uma API parceira demorar? Como empacotar, monitorar e escalar com zero downtime?*

Este material consolida todos os conceitos dessa jornada.

## 1. Fundamentos de Spring Boot, JPA & Código Limpo

### 1.1. Lombok Descomplicado (@Data, @NoArgsConstructor, @AllArgsConstructor)

O Lombok elimina código boilerplate repetitivo gerando métodos em tempo de compilação:

- **@Data (O Combo 5 em 1)**: Gera Getters, Setters, toString(), equals() e hashCode().
- **@NoArgsConstructor (Construtor Vazio)**: public Usuario() {}. **Por que é obrigatório?** O Jackson (conversor JSON) e o Hibernate (banco de dados) precisam de um construtor vazio para instanciar o objeto vazio antes de preencher os atributos.
- **@AllArgsConstructor (Construtor Cheio)**: Cria o construtor com todos os campos, facilitando a criação de objetos nos testes unitários em uma única linha.
- **A Pegadinha do Java**: Quando você cria qualquer construtor com parâmetros no Java, o compilador *apaga* o construtor vazio padrão. Por isso, a dupla **@NoArgsConstructor + @AllArgsConstructor** é usada sempre junta.

### 1.2. Por que usar Optional<T> nos Repositories?

Exemplo: Optional<Restaurante> findByNome(String nome);

- **A Analogia da Caixa**: O Optional é uma caixa protetora que pode vir com o objeto ou vazia.
- **Prevenção de NullPointerException**: Obriga o desenvolvedor a tratar a ausência de forma elegante e segura: `orElseThrow(() -> new EntityNotFoundException("Não encontrado"))` ou `isPresent()`.
- **Por que métodos que retornam List não usam Optional?** Porque uma lista nunca precisa ser null. Quando nenhum registro é encontrado, o Spring simplesmente retorna uma lista vazia [] com tamanho 0.

### 1.3. Gerenciamento de Transações (@Transactional)

Controla o ciclo de vida das transações com o banco (COMMIT no sucesso, ROLLBACK em caso de exceção).

- **readOnly = true em Consultas**: Desativa o *Dirty Checking* do Hibernate (não verifica se objetos mudaram na memória), economizando CPU e RAM. Em produção, direciona SELECTs para réplicas de leitura.
- **rollbackFor = Exception.class**: Garante rollback para qualquer tipo de erro (por padrão o Spring só desfaz para RuntimeException).
- **Propagação (Propagation)**: REQUIRED (usa a transação existente ou cria nova) vs REQUIRES\_NEW (cria sempre transação independente, ideal para salvar logs de auditoria mesmo se a transação principal falhar).

### 1.4. Auditoria de Dados: @PrePersist e Suas Alternativas

1. **@PrePersist (JPA Callback)**: Método executado automaticamente imediatamente antes do INSERT no banco.
2. **@CreationTimestamp / @UpdateTimestamp (Hibernate)**: Mais popular e limpo. Basta colocar a anotação diretamente sobre o atributo LocalDateTime dataCadastro.
3. **Valor Padrão no Java**: private LocalDateTime dataCriacao = LocalDateTime.now(); (simples, inicializa na memória).

4. **Spring Data JPA Auditing:** Padrao corporativo completo com `@CreatedDate`, `@LastModifiedDate` e `@CreatedBy` (armazena o usuario autenticado do Spring Security).

## 1.5. Spring Cache: Alta Performance em Memoria RAM

Evita consultas repetitivas ao banco de dados mantendo resultados frequentes na memoria RAM (tempo de resposta cai de 150ms para 0.5ms):

- **@EnableCaching:** Anotacao colocada na classe principal (DeliveryApiApplication) para ligar o motor de cache do Spring.
- **@Cacheable(value = 'produtos', key = '#id')**: Se o item ja estiver na memoria (Cache Hit), devolve direto e **nem executa o metodo Java** (zero queries no banco). Se nao estiver (Cache Miss), busca no banco e salva no cache.
- **@CacheEvict(value = 'produtos', key = '#id')**: Remove o item da memoria quando ele for alterado ou excluido, evitando que clientes vejam precos ou dados desatualizados.
- **Sintaxe (value = '...', key = '#...')**: O value e o nome da gaveta (ex: 'produtos'). O key e a etiqueta que deve bater com o nome do parametro (ex: #categoria, #id + '-' + #cep).
- **A Pegadinha do Import:** Use sempre `org.springframework.cache.annotation.Cacheable` (para metodos). O `jakarta.persistence.Cacheable` e exclusivo de classes `@Entity` e da erro em metodos.

## 2. Seguranca (JWT) & Documentacao Swagger (OpenAPI 3)

### 2.1. Como Testar Tokens JWT no Swagger UI

1. Execute POST `/api/auth/login` com credenciais validas e copie o token retornado no JSON.
2. No topo da pagina do Swagger UI, clique no botao verde **Authorize**.
3. Cole o token no campo de texto (nao digite 'Bearer', pois a configuracao HTTP Bearer no SwaggerConfig.java ja inclui o prefixo automaticamente).
4. O cadeado ficara com icone de autenticado e todas as rotas protegidas incluirao o header Authorization.

### 2.2. Como os Schemas OpenAPI sao Gerados e Onde Definir?

- **Geracao Automatica:** O SpringDoc inspeciona os controllers e monta a secao Schemas com base em classes presentes em `@RequestBody` e retornos de `ResponseEntity<T>`.
- **Onde customizar:** Diretamente nos **DTOs** usando `@Schema(description = '...', example = '...')` e validacoes do `jakarta.validation` (`@NotBlank`, `@Min`, etc., que geram asterisco vermelho de campo obrigatorio no Swagger).

## 3. Observabilidade & Monitoramento (A Triade de Ouro)

Em producao, a observabilidade responde: *como saber a saude do sistema sem precisar adivinhar?*

### 3.1. Health Checks (Spring Boot Actuator)

- **DatabaseHealthIndicator:** Testa a conexao real com o banco (`connection.isValid(1)`). Se o banco cair, responde DOWN.
- **ExternalServiceHealthIndicator:** Testa APIs externas essenciais (ex: Gateway de Pagamento, Correios).
- **Integracao com Kubernetes / Load Balancer:** O Kubernetes consulta o `/actuator/health` a cada 5s. Se responder DOWN, mata o container e sobe outro automaticamente!

### 3.2. Auditoria e Rastreabilidade de Logs (CorrelationIdFilter)

- **O Problema:** Milhares de requisicoes simultaneas misturam linhas de logs de diferentes clientes na mesma tela.
- **A Solucao (Correlation ID):** Cada requisicao ganha um UUID unico injetado no **MDC (Mapped Diagnostic Context)**. Todas as linhas de log geradas por aquela thread imprimirao automaticamente o mesmo ID!
- **Cabecalho X-Correlation-ID:** Devolvido na resposta HTTP para que o cliente informe o protocolo de erro ao suporte.

### 3.3. Metricas com Micrometer, Prometheus & Grafana

- **Micrometer:** O 'SLF4J das metricas'. Padroniza metricas e permite trocar de Prometheus para Datadog sem alterar o código Java.
- **Prometheus (Porta 9090):** Banco de series temporais que raspa as metricas da rota /actuator/prometheus.
- **Grafana (Porta 3000):** Interface visual com dashboards em tempo real conectada no Prometheus.

### 3.4. Distributed Tracing com Zipkin (Porta 9411)

- **Linha do Tempo Visual:** Mostra em formato de cascata (waterfall) quanto tempo cada etapa da requisicao demorou (banco, validacao, chamadas externas), identificando gargalos em segundos.
- **Trace:** A viagem completa da requisicao da ponta a ponta.
- **Span:** Cada etapa individual dentro da viagem (ex: uma query SQL ou chamada HTTP).

## 4. DevOps: Docker & Docker Compose Descomplicados

### 4.1. A Regra de Ouro das Portas: FORA : DENTRO

```
-p 3000:3000 -> [PORTA DO SEU PC (FORA)] : [PORTA INTERNA DO CONTAINER (DENTRO)]
```

- A porta da **esquerda** é onde você acessa no seu navegador (localhost:3000).
- A porta da **direita** é a porta interna real onde a aplicação escuta dentro da caixa do container.

### 4.2. Comunicacao entre Containers no Docker Compose

- **Nunca use localhost entre containers** (pois localhost aponta para dentro do proprio container).
- Na mesma rede bridge, **os containers conversam pelo NOME DO SERVICO e pela PORTA INTERNA:**
  - Prometheus busca metricas na API: `http://delivery-api:8080/actuator/prometheus`
  - Grafana consulta o Prometheus: `http://prometheus:9090`
  - API envia traces para o Zipkin: `http://zipkin:9411/api/v2/spans`
  - Nginx redireciona da porta 80 para a API: `http://delivery-api:8080`

### 4.3. Variaveis de Ambiente & Relaxed Binding

- Sintaxe no Spring: `jwt.secret=${JWT_SECRET:valor_padrao}` (usa variavel se existir, ou fallback).
- Arquivo `.env`: Padrao de mercado para armazenar senhas locais, sempre ignorado no `.gitignore`.
- **Relaxed Binding:** O Spring Boot converte automaticamente variaveis do Linux em propriedades Java (ex: `SPRING_DATASOURCE_URL` vira `spring.datasource.url`).

## 4.4. Colinha de Comandos Docker Compose

```
# Subir todos os containers em segundo plano compilando a API:
docker compose up -d --build

# Acompanhar logs de todos os servicos em tempo real:
docker compose logs -f

# Acompanhar apenas os logs da API Java:
docker compose logs -f delivery-api

# Parar todos os containers:
docker compose down

# Parar containers e limpar volumes persistentes (zera Grafana e banco):
docker compose down -v

# Parar e remover todos os containers do sistema (limpeza geral):
docker stop $(docker ps -aq) && docker rm $(docker ps -aq)
```

## 5. Visao de Futuro: Microservicos & Kubernetes

### 5.1. Como Quebrar um Monolito em Microservicos

- **Separacao por Contexto de Negocio (Bounded Contexts):** Cada grande modulo vira um projeto Spring Boot independente com seu proprio repositório e ciclo de deploy:
  - auth-service: Responsavel exclusivamente por cadastro, senhas e emissao de tokens JWT.
  - catalogo-service: Gerencia restaurantes, cardapios, produtos e precos.
  - pedido-service: Carrinho de compras, criacao e ciclo de vida do pedido.
  - pagamento-service: Comunicacao com adquirentes de cartao de credito e PIX.
  - notificacao-service: Disparo assincrono de e-mails, push notifications e SMS.
- **Regra Sagrada: 1 Banco de Dados por Servico (Database-per-service):** Jamais compartilhe a mesma base de dados. Se dois servicos acessam o mesmo banco, voce tem um 'Monolito Distribuido' (o pior dos dois mundos). Cada servico possui seu proprio schema/banco.

### 5.2. Comunicacao entre Microservicos: Sincrona vs. Assincrona

- **Sincrona (HTTP/REST via Spring Cloud OpenFeign):** Chamada direta entre servicos. Otima para consultas pontuais (ex: Pedido consulta se Produto existe no Catalogo). Cuidado com o acoplamento temporal: se o Catalogo estiver fora, o Pedido falha.
- **Assincrona Orientada a Eventos (RabbitMQ / Apache Kafka):** Desacoplamento total! Ao finalizar um pedido, o servico publica um evento na fila: '*PedidoCriado*'. Pagamento e Notificacao processam em segundo plano. Se o servico de e-mail cair, a compra nao e cancelada; a mensagem fica guardada na fila esperando o servico voltar.
- **API Gateway (Spring Cloud Gateway):** Porta de entrada unica para o aplicativo mobile e web. O cliente acessa apenas <https://api.delivery.com> e o Gateway roteia internamente para o microservico correto.

### 5.3. O Papel do Kubernetes (O Maestro da Nuvem)

O Docker cria e executa os containers individuais. O **Kubernetes (K8s)** e o orquestrador que gerencia centenas de containers distribuidos em dezenas de servidores fisicos na nuvem (AWS/GCP/Azure):

1. **Auto-Healing (Auto-Recuperacao):** Monitora o `/actuator/health`. Se um container travar, congelar ou responder DOWN, o K8s mata e sobe uma copia nova em 2 segundos sem intervencao humana.
2. **Auto-Scaling (HPA):** No horario de pico (ex: domingo as 12h), sobe automaticamente de 3 para 30 containers da API conforme o uso de CPU. Quando o movimento cai, encerra os extras para economizar dinheiro.
3. **Load Balancing (Distribuicao de Carga):** Recebe milhares de conexoes e distribui uniformemente entre todas as instancias ativas.
4. **Zero-Downtime Deploy (Rolling Updates):** Atualiza para a versao 2.0.0 substituindo um container por vez; o cliente nunca vê tela de 'sistema em manutencao'.

### 5.4. A Filosofia 'Monolith First' (Martin Fowler)

Grandes empresas (como Amazon, Netflix e Nubank) comecaram como monolitos. A recomendacao dos maiores arquitetos do mundo e iniciar com um monolito bem desenhado e modular (como este projeto Delivery API) e so quebrar em microservicos quando o time ou a escala exigirem.

## 6. Referencia Pratica: Portas, URLs & Endpoints

### 6.1. Tabela de Portas do Ecosystema Local (Docker)

| Servico                | Porta / URL Local                           | Finalidade & Acesso                                    |
|------------------------|---------------------------------------------|--------------------------------------------------------|
| Nginx (Proxy Reverso)  | http://localhost (Porta 80)                 | Portaria de entrada padrao web (sem porta no browser!) |
| Delivery API (Swagger) | http://localhost:8080/swagger-ui/index.html | Documentacao e testes interativos das rotas            |
| Actuator Health        | http://localhost:8080/actuator/health       | Status UP/DOWN do banco e servicos externos            |
| Prometheus             | http://localhost:9090                       | Banco de metricas raspadas da API                      |
| Grafana                | http://localhost:3000                       | Dashboards visuais (login: <i>admin / admin</i> )      |
| Zipkin                 | http://localhost:9411                       | Linha do tempo e analise de latencia das rotas         |
| H2 Database Console    | http://localhost:8080/h2-console            | JDBC: jdbc:h2:mem:deliverydb   User: sa                |

### 6.2. Resumo das Principais Rotas da Delivery API

| Modulo       | Metodo   | Endpoint                 | Descricao                                   | Seguranca  |
|--------------|----------|--------------------------|---------------------------------------------|------------|
| Auth         | POST     | /api/auth/register       | Cadastrar novo usuario no sistema           | Publica    |
| Auth         | POST     | /api/auth/login          | Autenticar usuario e obter token JWT        | Publica    |
| Clientes     | POST/GET | /api/clientes            | Cadastrar e listar clientes ativos          | Bearer JWT |
| Restaurantes | GET      | /api/restaurantes        | Listar restaurantes paginados e com filtros | Publica    |
| Restaurantes | POST/PUT | /api/restaurantes        | Cadastrar ou atualizar restaurante          | Bearer JWT |
| Produtos     | GET      | /api/produtos/{id}       | Buscar produto por ID ou categoria          | Publica    |
| Produtos     | POST/PUT | /api/produtos            | Cadastrar ou atualizar produto              | Bearer JWT |
| Pedidos      | POST     | /api/pedidos             | Criar e fechar novo pedido                  | Bearer JWT |
| Pedidos      | POST     | /api/pedidos/calcular    | Simular total de itens e frete              | Bearer JWT |
| Pedidos      | PATCH    | /api/pedidos/{id}/status | Atualizar maquina de estados do pedido      | Bearer JWT |

**Mensagem Final: Continue pensando como Engenheiro!**

O entendimento de como uma aplicacao se comporta em producao e o divisor de aguas na carreira de qualquer desenvolvedor backend. Parabens pelo dominio dessa stack completa: Clean Code, Seguranca JWT, Docker, Observabilidade (Prometheus/Grafana/Zipkin) e Arquitetura de Microservicos!